Data & Knowledge Engineering 159 (2025) 102452

Contents lists available at ScienceDirect

Data & Knowledge Engineering

journal homepage: www.elsevier.com/locate/datak

Conceptual design of multidimensional cubes with LLMs: An investigation

Stefano Rizzi *, Matteo Francia, Enrico Gallinucci, Matteo Golfarelli

DISI, University of Bologna, Bologna, Italy

ARTICLE INFO

Keywords:
Data warehouse design
Multidimensional model
Large Language Models
ChatGPT

ABSTRACT

Large Language Models (LLMs) can simulate human linguistic capabilities, thus producing a disruptive impact across several domains, including software engineering. In this paper we focus on a specific scenario of software engineering, that of conceptual design of multidimensional data cubes. The goal is to evaluate the performance of LLMs (precisely, of ChatGPT-4o) in multidimensional conceptual design using the Dimensional Fact Model as a reference. To this end, we formulate nine research questions to (i) understand the competences of ChatGPT in multidimensional conceptual design, following either a supply- or a demand-driven approach, and (ii) investigate to what extent they can be improved via prompt engineering. After describing the research process in terms of base criteria, technological setting, input/output format, prompt templates, test cases, and metrics for evaluating the results, we discuss the output of the experiment. Our main conclusions are that (i) when prompts are enhanced with detailed procedural instructions and examples, the results produced significantly improve in all cases; and (ii) overall, ChatGPT is better at demand-driven design than at supply-driven design.

1. Introduction

Thanks to their capability of learning from huge corpora, *Large Language Models* (LLMs) have shown to be an excellent tool for simulating human linguistic capabilities and generating credible text, thus producing a disruptive impact across several domains [1]. Software Engineering is among the fields in which the potential benefits of the LLM revolution are more obvious [2]. LLM-based approaches are currently being proposed for all steps in the software development life-cycle, including requirement elicitation, code generation, and refactoring [2–4].

Conceptual design is a key step in the engineering of software systems, since it determines their fundamental characteristics. Creativity is particularly relevant at this stage, and designers are recommended to produce and assess various solutions [5]. Some experiments in using LLMs for conceptual design have been conducted (e.g., [6,7]); it emerged that, although they can effectively support designers in this task by quickly providing draft solutions, a significant human involvement is still required to ensure the correctness of the result [3]. Several factors may affect the effectiveness of this process, among which the size and complexity of the specific domain to be analyzed, and whether the LLM has been trained or not on that domain.

In this paper we focus on a specific scenario, that of conceptual design of multidimensional data cubes which lie at the core of data warehouse and OLAP (OnLine Analytical Processing) systems. As compared to conceptual design of generic software systems, and even of operational databases, this scenario appears to be less difficult due to the peculiar constraints posed by

* Corresponding author.
E-mail addresses: stefano.rizzi@unibo.it (S. Rizzi), matteo.francia@unibo.it (M. Francia), enrico.gallinucci@unibo.it (E. Gallinucci), matteo.golfarelli@unibo.it (M. Golfarelli).

https://doi.org/10.1016/j.datak.2025.102452
Received 29 November 2024; Received in revised form 7 April 2025; Accepted 24 April 2025
Available online 7 May 2025
0169-023X/© 2025 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

the multidimensional model (e.g., in the structure of dimension hierarchies). Precisely, the goal of this paper is to evaluate the performance of LLMs in multidimensional conceptual design using the Dimensional Fact Model (DFM [8]) as a target formalism. To this end, we formulate nine research questions aimed at achieving two research goals, namely, understanding the prior competences of LLMs in multidimensional conceptual design and investigating to what extent they can be improved via prompt engineering. This is done with reference to the two main approaches to the design of multidimensional cubes, namely, supply- and demand-driven design.

As often done in the related literature, as a representative LLMs application for our investigation we chose ChatGPT. ChatGPT, powered by OpenAI [9], has become quite popular due to its capabilities in natural language generation and seamless user interaction [10]. In particular, for the experiments we used model GPT-4o.

The remainder of the paper is structured as follows. After discussing the related literature in Section 2, in Section 3 we describe in detail the investigation we conducted by explaining how we designed the experiments and answering the research questions. Finally, we draw the conclusions and outline the future work in Section 4.

2. Related work

2.1. LLMs for the software life-cycle

In the last couple of years, a lot of research and experimental work has been devoted to assessing the capabilities of LLMs in dealing with the software development life-cycle (see [1] for a comprehensive survey). For instance, the capabilities of LLMs for supporting non-expert users in drawing three types of UML (Unified Modeling Language) diagrams are explored in [11]. It turns out that, while LLMs are good in recognizing specific entities from natural language, their modeling of inter-entity relationships is often inaccurate. On the other hand, it is observed that the specified output format affects the performance of LLMs; specifically, hybrid-created diagrams the user may obtain starting from the raw output of the LLM are those ranked as the most accurate. On the other hand, the authors of [12] focus on how to build UML class diagrams enriched with OCL (Object Constraint Language) constraints using ChatGPT. They conclude that ChatGPT is only able to deal with small models, unable to properly handle some basic modeling concepts, and frequently producing obvious errors; besides, the results obtained from repeated tests are variable and inconsistent. Interestingly, *prompt patterns* (i.e., reusable prompt designs) are introduced in [3] as a way to improve software quality attributes when working with LLMs for software engineering. Specifically, they propose a catalogue of 13 prompt patterns for requirements elicitation, system design, code quality control, and code refactoring. The authors conclude that, although these prompt patterns can help mitigate the tendency of LLMs to hallucinate, significant human involvement is still required to ensure the correctness of the result of common software engineering tasks. Besides, none of these pattern is related to data design, being mostly focused on code.

With specific reference to conceptual design, an experiment to use LLMs for generating specifications from requirements documents, in the field of smart devices, is described in [7]. The authors argue that the core competence of conceptual design, namely, to select and justify the best suited solution to fulfill the requirements, is still missing; however, they recognize that in later stages of the development process, such as generating class diagrams from detailed implementation specifications and producing source code, LLMs prove to be quite effective. Some experiments in using ChatGPT for conceptual modeling (using for instance UML class diagrams and E/R diagrams) are discussed in [6]. The authors observe that a first draft diagram can be quickly generated by ChatGPT starting from a natural language description; however, a significant modeling experience is still required to refine and validate the results. Thus, they conclude that ChatGPT can support experts by streamlining conceptual design — but clearly it cannot replace them. In [10], the authors report on an experiment made with ChatGPT and conclude that the incorporation of LLMs in human-driven conceptual design does not have a significant impact on outputs, but it significantly shortens the design completion time entailing a smaller number of design steps.

A number of conceptual design solutions generated by an LLM are compared to a baseline of crowdsourced solutions in [13]. It is observed that, in the average, the LLM-generated solutions are more feasible, while the crowdsourced ones are more novel. Remarkably, it is also suggested that, when *few-shot learning* is adopted (i.e., the prompt is augmented with some examples of the task), the LLM-generated solutions become more similar to the crowdsourced ones.

The advantages of using LLMs to enhance morphological analysis in conceptual design are investigated in [14]. The experiments show that LLMs equip designers with cross-disciplinary expertise and enable structured deconstruction and examination of design challenges. Smart prompt engineering and close collaboration between LLMs and designers are recommended for best results. An analysis of how users interact with LLMs during conceptual modeling is made in [15], with reference to use case and domain modeling. The main findings concern the need for specific prompt templates to support users on the one hand, the adoption of a recommender to suggest relevant prompts or actions on the other.

In [16], the findings of some experiments made in using ChatGPT for metamodeling are discussed. It turns out that analyzing metamodels is difficult due to their size and complexity. Generating a program to transform a metamodel into a UML class diagram could be done accurately, but directly generating model instances from a given metamodel was problematic mainly due to the nondeterministic nature of ChatGPT.

We finally mention that, beyond text, some LLMs can also handle and generate content of other forms, e.g., images; these are known as *multimodal* LLMs, and GPT-4V and Gemini (Pro/Ultra) are two examples. An attempt to employ multimodal LLMs in software engineering has been done in [17], where images of hand-drawn UML class diagrams are converted into the corresponding diagram in PlantUML notation.

2.2. Conceptual design of multidimensional cubes

The approaches to multidimensional conceptual design can be classified into four categories: *supply-driven* (or data-driven), *demand-driven* (or requirement-driven), *mixed*, and *query-driven*. Supply-driven approaches design conceptual schemata starting from the data sources (described for instance through relational schemata); user requirements impact on design by allowing the designer to select which parts of data are relevant for decision-making and by determining their structuring according to the multidimensional model [18]. Demand-driven approaches start from determining the business requirements of end-users, and how to map these requirements onto the available data sources is investigated only afterwards [19]. Mixed approaches combine data- and requirement-driven ones; in this case, both data source schemata and user requirements are used at the same time [20]. Finally, in query-driven approaches, a multidimensional schema is created starting from the set of OLAP queries the end-users are willing to formulate; these queries are specified using either SQL statements [21], MDX (Multidimensional Expressions) statements [22], pivot tables [23], or query trees [24].

In this work we consider supply-driven and demand-driven approaches, the two most common ones, which are quite different from the point of view of the competences they require. Supply-driven approaches can partially be automated; indeed, a draft conceptual schema can be obtained by following the functional dependencies present in the source schema, to be later (manually) refined based on the user requirements. Thus, this approach mainly relies on the application of an algorithm. On the other hand, demand-driven approaches are based on the correct understanding and representation of the user requirements by the designer; as such, besides a deep knowledge of the multidimensional model, they require a lot of experience – and possibly some knowledge of the business domain – from the designer.

3. The research process

As already stated, this work aims at assessing the performance of ChatGPT in multidimensional conceptual design. More precisely, we pursue two research goals:

RG.1: Evaluate the basic competences of ChatGPT in multidimensional conceptual design, with reference to both supply-driven and demand-driven approaches.
RG.2: Understand to what extent these competences can be improved via prompt engineering.

For simplicity, we take as a reference a basic form of the DFM including only the constructs of fact, measure, dimension, and attribute. In this form, a DFM schema is a graph whose root is the *fact* (represented as a box with the fact name —e.g., SALES— followed by a list of *measures* —e.g., Amount), whose other nodes are *attributes* —e.g., Product— represented as circles and connected by arcs representing many-to-one *roll-up relationships*, i.e., functional dependencies (FDs, for instance, Product → Category). Advanced constructs of the DFM (e.g., descriptive attributes, cross-dimensional attributes, multiple arcs, etc.) are not taken into account at this stage. Besides, the editing step of supply-driven design (where the initial DFM schema obtained by following the FDs in the source schema is adjusted by dropping non-interesting attributes) is not considered and will be a subject of future research.

For reproducibility purposes, the reader can visit our GitHub repository at https://github.com/big-unibo/llm4dfm, where we have published all the code to systematically run the experiments, including all definitions of test cases, used prompts, and DFM schemata obtained in output.

3.1. Experiment design

We designed our experiment in seven steps: (1) formulation of research questions; (2) set up of base criteria; (3) technological setting; (4) choice of input/output formats; (5) definition of prompt templates; (6) definition of test cases; (7) definition of criteria for evaluating the results. These steps are explained in the following subsections.

3.1.1. Research questions

To pursue our research goals, we formulate the following research questions (RQs):

RQ.1: Does ChatGPT know what conceptual design of multidimensional cubes is and how to perform it?
RQ.2: Does ChatGPT know the difference between supply-driven and demand-driven design?
RQ.3: How does ChatGPT perform in supply-driven design, i.e., when generating multidimensional conceptual schemata starting from operational source schemata?
RQ.4: How does ChatGPT perform in supply-driven design when prompt engineering is adopted?
RQ.5: How does ChatGPT perform in demand-driven design, i.e., when generating multidimensional conceptual schemata starting from end-user requirements?
RQ.6: How does ChatGPT perform in demand-driven design when prompt engineering is adopted?
RQ.7: Does the performance in demand-driven design depend on how the end-user requirements are expressed?
RQ.8: Does ChatGPT perform better when designing nodes or arcs of the DFM schema?
RQ.9: Does ChatGPT perform better in supply-driven or in demand-driven design?

Note that, while RQ.1 and RQ.2 explore what ChatGPT *knows*, the remaining ones explore what it can *do*.

The relationships between research goals and RQs are summarized in Fig. 1.

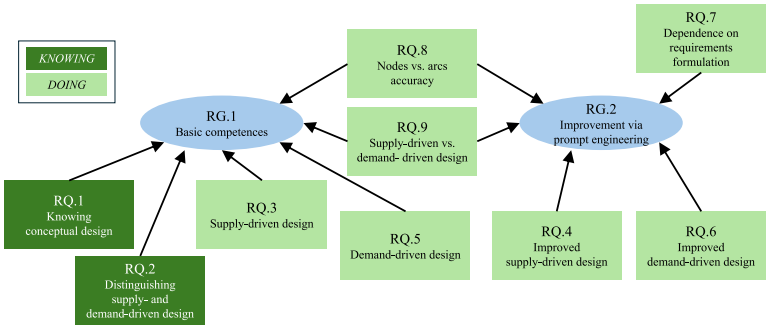

Fig. 1. Relationships between research goals (in cyan) and RQs.

3.1.2. Base criteria

The criteria we follow for our experiment are listed below:

- **Learning**. We adopt a prompt-based learning method, which involves giving training examples in input to the LLM and is often used as an alternative to *fine-tuning*, which is based on parameter updates [25]. Specifically, we adopt *0-shot learning* for RQ.3 and RQ.5 (which investigate the basic capabilities of ChatGPT) and *few-shot learning* for RQ.4 and RQ.6 (which imply prompt engineering).[1] While 0-shot operates with no examples of the task at hand, few-shot learning relies on some training examples (in addition to the task description and input) [26]. Only using input/output pairs in the examples may not be sufficient to learn a complex task. The *chain-of-thought* technique [27] has been shown to improve learning by including a list of reasoning steps in the examples. We adopt chain-of-thought learning, in combination with few-shot learning, for RQ.4 and RQ.6.
- **Reproducibility**. A major challenge when working with LLMs is the lack of reproducibility of the experiments due to their non-deterministic nature [12]. To mitigate this issue, we set the temperature parameter to 0.1 for all chats.
- **Domain**. It is recognized that the problem domain is important for LLMs; the more an LLM knows about the domain, the better the model it produces [12]. All the cases we submitted described real domains — some well-known (e.g., purchases), some less common (e.g., crossfit workouts).
- **Conversation-awareness**. The answers obtained from ChatGPT may depend heavily on the previous questions asked during a conversation. Thus, as also suggested in [12], we start a new chat for each case. Specifically, the prompts for each test are submitted to ChatGPT 10 times in separate chats, and the results are then either manually integrated (for the prompts giving a textual answer, i.e., those used in RQ.1 and RQ.2) or averaged (in terms of evaluation metrics, for the prompts whose answer is a DFM schema, i.e., those in the remaining RQs).
- **Iteration**. At this stage of investigation, the first answer obtained is considered and no attempt is made to improve it by further prompting ChatGPT with suggestions and hints.

3.1.3. Technological setting

Experiments have been carried out on the ChatGPT-4o model, deployed on a paid Azure subscription. To query the model systematically, we have developed a Python application that automatically submits prompts via APIs and computes the evaluation metrics by comparing the DFM schemata obtained with the corresponding ground truths (as explained in Section 3.1.7). Running all 9 test cases for 10 times takes around 30 min for each RQ. To support a manual inspection of the results, a simple HTML+Javascript GUI has also been developed to draw and compare the DFM schemata obtained by ChatGPT and those in the ground truth.

3.1.4. Input/output format

RQ.3 and RQ.4 require that a source relational schema is given in input to ChatGPT for the task. Since ChatGPT is well familiar with SQL, we express the input schemata via the SQL DDL (Data Definition Language), i.e., using `create table` statements, with no additional training required. RQ.5 and RQ.6 require a set of requirements in input; these are expressed in natural language with no particular format.

As to the output, all our RQs (except RQ.1 and RQ.2) require that a DFM schema is delivered. As the output format we adopt *YAML* (YAML Ain't Markup Language), a human-readable data serialization language commonly used for configuration files and in applications where data are being stored or transmitted.[2] ChatGPT knows YAML, so no additional training about the language syntax is required; however, it must be instructed about the specific tags we introduced, namely:

- *name*, to denote fact, measure, and attribute names;

[1] RQ.1 and RQ.2 are not listed here because they rely on explorative questions not involving any learning.
[2] https://yaml.org/spec/history/2001-08-01.html.

- *fact*, to denote the fact;
- *measures*, to introduce the list of measures;
- *dependencies*, to introduce the list of FDs;
- *from* and *to*, to denote the start and end nodes of each FD;
- *role* (optional), to denote the role of an FD.

3.1.5. Prompt templates

Following the suggestions given in [2,3], the prompts we adopt during our chats are structured according to some templates:

- *Investigation prompts*. These prompts are used in RQ.1 and RQ.2 to understand what the LLM knows. They are structured as simple questions, e.g., for RQ.1:

  Prompt: Which models for conceptual design of multidimensional cubes do you know?

- *Instruction prompts*. These are used in all other RQs, aimed at understanding what ChatGPT can do, to explain how to perform a task. Their structure includes:

  1. ROLE: the specific role assigned to ChatGPT to provide a context for the task (in all our tests, "You are a data warehouse designer").
  2. PROCEDURE: the task and the method suggested to perform it; for instance, for RQ.3:

     Prompt: Carry out supply-driven conceptual design of a multidimensional cube using the DFM (Dimensional Fact Model), starting from the following source relational schema.

     and for RQ.4:

     Prompt (excerpt): Your task is to create the conceptual schema for a multidimensional cube in a supply-driven fashion, i.e., starting from a source relational schema D. To this end, the first step is to create a directed acyclic graph G where each node corresponds to a (simple or composite) attribute of D and each arc corresponds to a functional dependency (FD). You can do this via the following steps:
     0. Find a fact R. A fact is a relation that has some temporal or date attributes and includes some foreign keys. The more foreign keys R has, the better candidate it is to be a fact. The primary key of a fact is often a composite one. If there is a relation S that includes a foreign key referencing R, then most probably S is a better candidate than R to be a fact.
     1. Initialize […]
     2. Expand R […]
     3. Rearrange G […]
     4. Rename the fact […]
     5. Choose measures […]
  3. EXAMPLE: an example of a test case, an explanation of the procedure suggested to solve it (following the chain-of-thought approach), and the expected output. Either 0 or 3 examples are provided in our tests.
  4. OUTPUT FORMAT: how the output (i.e., the DFM schema) should be delivered. The prompt we use for all tests is the following:

     Prompt: The output I expect is a DFM schema in YAML formatted as follows: (1) the fact is a "fact" tag including a "name" tag; (2) all measures are listed inside a "measures" tag, each is an empty item containing a "name" tag; (3) all functional dependencies between attributes in a hierarchy are listed inside a "dependencies" tag: each is an empty item containing a "from" tag, listing the finer attribute, a "to" tag, listing the coarser attribute, and optionally a "role" tag; (4) the "dependencies" list also includes an item from the fact to each dimension, and one from the fact to each measure; (5) if a dependency is from and/or to a combination of attributes, they should be comma-separated.

- *Case prompts*. These are used to assign a specific design task to ChatGPT. Their structure includes:

  1. TASK: the specific task. For instance, for RQ.3:

     Prompt: Carry out supply-driven conceptual design of a multidimensional cube using the DFM (Dimensional Fact Model), starting from the following source relational schema.
  2. INPUT: the input of the task, namely, either a source relational schema in RQ.3 and RQ.4, or a set of requirements in RQ.5 and RQ.6. For instance, the requirements for test case C1 in RQ.5 are stated as follows:

     Prompt: Decision-makers are interested in analyzing the software installed on computers in a laboratory on a daily basis. Each computer has a model and a brand, each brand is supplied by a supplier who lives in a city at a given address. A software has a description and a type.
  3. OUTPUT: the output required; for instance, for RQ.3:

     Prompt: Return only the YAML without any further information or explanation.

Table 1
Test cases used for the experiment.

Case Id	Domain	♯ Tables/Attr.	♯ Tokens	Hidden FDs	Composite FKs	Fact ambiguity	Shared Hier.	Cycles
C1	Sw installations	5/14	49	N	N	N	N	N
C2	Purchases	5/19	54	Y	N	N	N	N
C3	Card purchases	5/34	94	Y	N	N	Y	N
C4	Car races	7/33	65	N	Y	N	N	N
C5	Crossfit	7/30	71	Y	Y	Y	N	N
C6	Staff recruitment	6/21	81	N	N	N	Y	Y
C7	Car rentals	8/35	106	Y	Y	Y	Y	N
C8	Train trips	9/38	130	Y	Y	Y	Y	N
C9	Flights	10/50	179	Y	Y	Y	Y	N

```
PURCHASES(PurchaseCode,Date,Amount,Currency,Exchange,CCNumber:CREDIT_CARDS,StoreCode:STORES)
CREDIT_CARDS(CCNumber,TypeCode:CARD_TYPES,HolderCode:HOLDERS,ExpirationDate,CreditLimit)
HOLDERS(HolderCode,FirstName,LastName,BirthDate,Job,IncomeRange,Quality)
CARD_TYPES(TypeCode,Circuit,IssuingBank,Address,Country,CardName,Color)
STORES(StoreCode,Name,Address,Country,Area,WebSite,Email,Type)
```

Decision-makers are interested in analyzing the amount and exchange of purchases made in stores using credit cards on a day-by-day basis. A store has a name, an address, a type, and is located in a country of an area. Credit cards have a type, a holder, an expiration date, and a credit limit. Each card type is described by a circuit, an issuing bank (with its address and country), by a card name, and by a color. A holder has a first name, a last name, an age, a job, and an income range.

Fig. 2. Source relational schema (top) and natural language requirements (bottom) for C3.

3.1.6. Test cases

We created a set of 9 test cases with increasing difficulties, based on several exercises in supply-driven design assigned to the students of a master course in Business Intelligence over the years. Each exercise provided a source relational schema; from this schema, a basic DFM schema was created using the FD-chasing algorithm in [8] and adding the missing FDs in case the source schema was not normalized (e.g., from City to Region), then the requirements in natural language were manually written by reverse engineering from that DFM schema.

Table 1 lists the test cases with their business domain, the number of tables and attributes in the source schemata, and the number of tokens in the corresponding natural language requirements; the remaining columns state whether some hidden FDs are present in the source schema, whether there the source schema shows composite foreign keys (FKs), whether there is an ambiguity in choosing the fact, whether shared hierarchies are involved (e.g., ShipmentDate and DeliveryDate), and whether cycles (recursive hierarchies) are involved.

An example of test case is shown in Fig. 2; in the source schema, primary keys are underlined while foreign keys are followed by ":" and by the name of the referenced relation.

3.1.7. Evaluation of the results

To investigate the RQs related to what ChatGPT knows, i.e., RQ.1 and RQ.2, we use prompts that give a textual answer. In this case, the results are evaluated by having an expert analyze these answer and measure the percentage of correct assertions they include. The resulting percentage is then mapped onto a qualitative evaluation as follows: "no" below 25%, "yes" over 75%, and "to some extent" between 25% and 75%.

The remaining RQs are aimed at assessing what ChatGPT can do, and the prompts used to investigate them return DFM schemata. To evaluate each DFM schema generated by ChatGPT we compare it with the corresponding ground truth in YAML format:

- As to supply-driven design, the ground truth for each test case was generated by applying the FD-chasing algorithm proposed in [8] to the source schema; the fact was chosen as the source table which allows for including in the DFM schema the largest number of attributes of the source schema. Since the FD-chasing algorithm is deterministic and, among our test cases, there are no ties in the choice of the fact, exactly one ground truth is determined for each test case.
- As to demand-driven design, the ground truth was created manually by an expert starting from the requirements.

Fig. 3 shows both ground truths for test case C3, as derived from the source schema and the requirements in Fig. 2, respectively.

As an evaluation metrics we adopt the *F1 score*, which is commonly used for graph comparison [28,29]. The F1 score is defined as the harmonic mean of precision and recall; thus, it symmetrically represents precision and recall in one metric, reaching its best and worst values at 1 and 0, respectively. When comparing two DFM schemata, we treat them as graphs and compute the F1 score for both nodes and arcs. Note that node comparison is made in terms of both names and tags (thus, two nodes with the same name match only if either they are both measures, or none of them is a measure); arcs are compared based on their ends and their roles (if any).

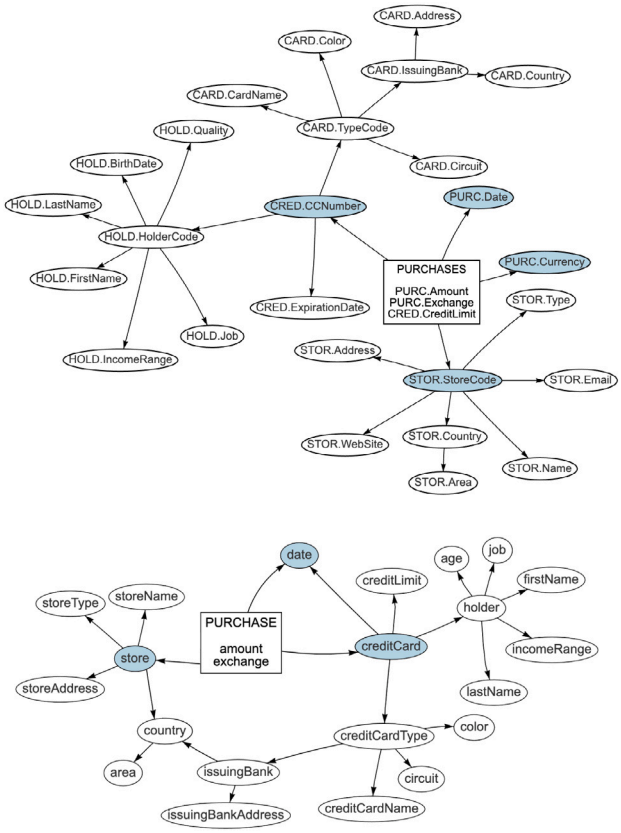

Fig. 3. Supplier-driven (top) and demand-driven (bottom) ground truths for C3.

A problem we met in the computation of these metrics is that of name matching. In supply-driven design, the names used in the DFM schemata compared should exactly take the form RELATION_NAME.AttributeName (e.g., REGION.Name), so they should match exactly. Conversely, in demand-driven design, the names chosen by ChatGPT may be correct even if they are not exactly the same we chose for the ground truth. To cope with this, for each test case we prepared a list of names that should be considered equivalent when comparing demand-driven DFM schemata (e.g., CreditCardType, CardType, and Type).

One more issue related to demand-driven design is that of the count measure. In principle, this measure should be present only when the granularity of the fact is coarser than the one of the source database. Unfortunately, understanding the granularity based on the end-user requirements only is not simple even for a human designer. For this reason, we chose to remove count measures from all the ground truths of demand-driven design, and we do not consider the count measures possibly created by ChatGPT when comparing DFM schemata.

To answer RQ.3, RQ.4, RQ.5, and RQ.6, which require a qualitative evaluation of performances, we average the F1 scores for nodes and arcs across all test cases. An average F1 score above 0.9 is considered excellent, one between 0.8 and 0.9 is considered good, one below 0.5 is considered poor; if the F1 score is 0.5 or more and less than 0.8, then the performance is considered to be average.

3.2. Answer to RQ.1: Knowing conceptual design

This session was aimed at assessing the knowledge ChatGPT has of conceptual design, and it only uses investigation prompts. The answers reported below are the result of an integration of the answers obtained by different runs; for the sake of brevity, only the main parts of the answers are included, omitting the details (for instance, those about the features of each model).

Prompt: Which models for conceptual design of multidimensional cubes do you know?

Answer (excerpt): (1) Star Schema Model; (2) Snowflake Schema Model; (3) Galaxy Schema (Fact Constellation) Model; (4) Data Vault Model; (5) Anchor Modeling; (6) Corporate Information Factory; (7) Dimensional Fact Model (DFM); (8) Multidimensional Entity-Relationship (MER) Model; (9) Object-Oriented Multidimensional Model (OOMD).

The answer to this question denotes some confusion — which is indeed not surprising, because the online literature by practitioners (e.g., discussion forums) tends to mix models at different levels. Indeed, the first three models are definitely logical ones [30]. The *data vault model* is defined at both the conceptual [31], logical [32], and physical [33] levels; it is suitable to model multi-source (operational) environments needing a fast adaptation to changes, but not for conceptual design of multidimensional cubes since it does not explicitly represent multidimensional concepts (e.g., hierarchies). The same holds for *anchor modeling* [34]. The *Corporate Information Factory* is in fact a data warehouse architecture, not a conceptual model [35]. The *DFM* [8] and the *MER model* [36] are well-known conceptual models for data warehouse design. Finally, different *OOMD models* have been proposed in the literature (e.g., [37–40]), with small variants. Among the latter models, in this paper we focus on the DFM, which is widely adopted in the research literature. Overall, out of 9 models mentioned by ChatGPT, only 3 were actually conceptual ones; thus, the correctness of this answer can be evaluated to 3/9.

> **Prompt**: What are the benefits of conceptual design when designing multidimensional cubes?
>
> **Answer (excerpt)**: (1) Clarity and Communication; (2) Improved Data Quality and Consistency (error reduction, standardization); (3) Better Planning and Documentation; (4) Maintainability and Scalability; (5) Efficient Query Performance; (6) Cost and Time Efficiency (reduced development time, resource optimization); (7) Alignment with Data Governance and Compliance; (8) Support for Advanced Analytics; (9) User Empowerment (self-service BI, intuitive data exploration). Overall, conceptual design is a critical step in the development of multidimensional cubes, providing a solid foundation for a robust, efficient, and scalable data warehouse solution.

This answer comprehensively list the key benefits of conceptual design. Since efficiency (point 5) is in fact determined by logical design rather than by conceptual design, the correctness of this answer can be evaluated to 8/9.

> **Prompt**: How would you design a multidimensional schema using the DFM?
>
> **Answer (excerpt)**: Steps to draw a DFM schema: (1) Identify the Business Process; (2) Identify the Fact Table; (3) Identify the Dimensions; (4) Define Hierarchies within Dimensions; (5) Draw the Fact Table and Dimension Tables; (6) Include Attributes in Dimension Tables.
>
> Example of a DFM Schema: Sales Analysis (see Fig. 4, left). This visual diagram places the fact table in the center with lines connecting to the surrounding dimension tables. Each dimension table contains attributes that define the context for the measures in the fact table. This provides a clear and organized way to design and understand the multidimensional schema for a data warehouse.

This answer is largely incorrect. Both the terminology used (steps 2, 5, and 6) and the resulting diagram (shown in Fig. 4, left) actually refer to a star schema, not to a DFM schema. Note that, while the true DFM schema (Fig. 4, right) shows the roll-up order of attributes within hierarchies, the star schema does not, since all the attributes in each hierarchy are stored together in a single denormalized dimension table. We also note that the semantics of colors adopted by ChatGPT to draw the diagram is unclear (we suppose keys are meant to be in blue, but this is not done consistently). Overall, we evaluate the correctness of this answer to 0.25 (average of 3/6 for the steps and 0 for the example).

Discussion. From these experiments we can conclude that ChatGPT knows the benefits of conceptual design but fails in correctly classifying some models as conceptual, and tends to confuse a DFM schema with a star schema. The average evaluation for the three answers is 0.49; thus, the answer to RQ.1, "Does ChatGPT know what conceptual design of multidimensional cubes is and how to perform it?", is **"to some extent"**.

3.3. Answer to RQ.2: Distinguishing supply- and demand-driven design

This session was aimed at understanding if ChatGPT can correctly distinguish between supply- and demand-driven design; as for RQ.1, it only uses investigation prompts and the answers reported are the result of an integration of the answers obtained by different runs.

> **Prompt**: What's the difference between supply-driven and demand-driven conceptual design of a multidimensional cube?
>
> **Answer (excerpt)**: They are two distinct methodologies for the conceptual design of multidimensional cubes in data warehousing. Each has its own focus and process for designing the data structures needed for effective data analysis.
>
> - *Supply-Driven Conceptual Design*. Focus: available data sources. Process: (i) Analyze existing data sources; (2) Identify key data elements; (3) Model the schema based on available data; (4) Iterate with data availability. Advantages: quick implementation, consistency with existing data sources, resource utilization. Disadvantages: may not meet all user needs, limited flexibility.
> - *Demand-Driven Conceptual Design*. Focus: user requirements and analytical needs. Process: (1) Gather user requirements; (2) Define key business metrics; (3) Design schema to meet requirements; (4) Identify data sources to meet design. Advantages: user-centric, flexibility (with regards to user needs), targeted analysis. Disadvantages: potentially longer implementation, data availability.

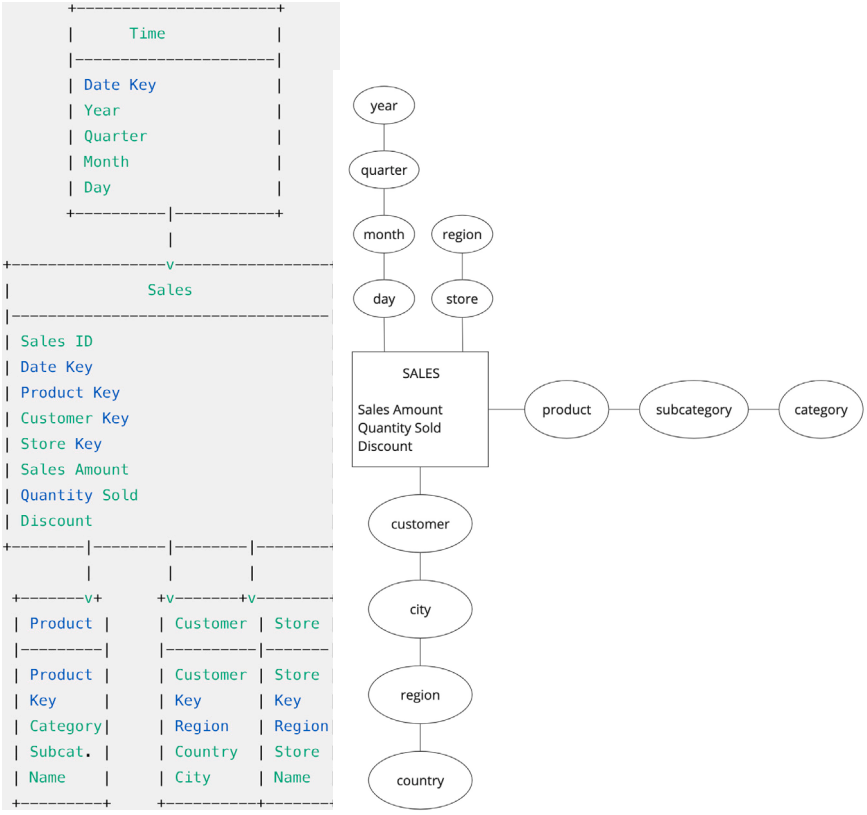

Fig. 4. A DFM schema according to ChatGPT (left) and the corresponding real DFM schema (right). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Although no precise indications are given about how to carry out the design process in practice, the differences between the two approaches and their pros and cons are correctly described. Thus, we evaluate the correctness of this answer to 100%.

When explicitly prompted about the process for supply-driven design, ChatGPT gives a more precise answer.

> **Prompt**: How do you carry out supply-driven conceptual design of a multidimensional cube starting from a source relational schema using the DFM?
>
> **Answer (excerpt)**: (1) Analyze Source Schema; (2) Identify Fact Candidates; (3) Identify Dimension Candidates; (4) Define Facts and Measures; (5) Define Dimensions and Hierarchies; (6) Create the DFM Schema; (7) Map Source Schema to DFM; (8) Design the Fact Table; (9) Design the Dimension Tables; (10) Validate the Model; (11) Refine the Model; (12) Implement the Physical Model; (13) Populate the Data Warehouse.

Although this gives no practical indications about how to carry out steps (6) and (7) (i.e., the only ones specifically related to the DFM), there are no errors; thus, we evaluate the correctness to 100%.

Similarly for demand-driven design:

> **Prompt**: How do you carry out demand-driven conceptual design of a multidimensional cube starting from the end-user requirements using the DFM?
>
> **Answer (excerpt)**: (1) Requirements Gathering (interviews, questionnaires, workshops, existing reports); (2) Identify Fact Tables; (3) Identify Dimensions; (4) Create a DFM Schema; (5) Define Facts and Measures; (6) Define Dimensions and Hierarchies; (7) Design the Fact Table; (8) Design the Dimension Tables; (9) Validate the Model with End-Users (prototyping, feedback, iteration); (10) Refine the Model; (11) Implement the Physical Model; (12) Populate the Data Warehouse.

Even in this case, no precise indication is given about step (4), which involves the DFM; again, we evaluate the correctness to 100%.

Discussion. From these experiments we can conclude that ChatGPT has a precise idea of the distinction between supply- and demand-driven approaches, and some generic knowledge of the corresponding design processes. Based on the evaluation made of the three answers, we conclude that the answer to RQ.2, "Does ChatGPT know the difference between supply-driven and demand-driven design?", is **"yes"**.

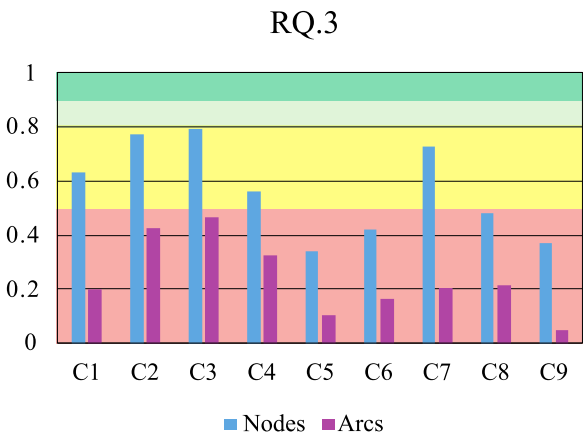

Fig. 5. F1 score for nodes and arcs of the DFM schemata obtained for each test case (supply-driven design; red means poor, yellow means average, light green means good, green means excellent). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.4. Answer to RQ.3: Supply-driven design

To understand what ChatGPT can actually do in supply-driven design, we fed it with all the test cases summarized in Table 1. To this end we use, for each test case, a simple instruction prompt with no specific procedure, just a request to carry out supply-driven design followed by a case prompt specifying a source relational schema. No example is provided.

> **Prompt (excerpt)**: Carry out supply-driven conceptual design of a multidimensional cube using the DFM (Dimensional Fact Model), starting from the following source relational schema:
>
> CREATE TABLE SUPPLIERS (SupplierName VARCHAR(50) PRIMARY KEY, SupplierCity VARCHAR(50), SupplierAddress VARCHAR(100));
>
> […]
>
> The output I expect is a DFM schema in YAML formatted as follows: […]. Return only the YAML without any further information/explanation.

The results are summarized in Fig. 5, in terms of the accuracy metrics introduced in Section 3.1.7. Fig. 6 allows to compare a DFM schema created by ChatGPT with the ground truth for test case C1 (the simplest one). Even for a very simple source schema featuring 5 tables, ChatGPT correctly identifies the fact but (i) creates the DFM schema as a non-connected graph; (ii) cannot correctly interpret the FDs entailed by the source schema (especially those related to composite keys); (iii) classifies the installation date as a measure rather than as a dimension. The worst results are obtained for C5 (possibly because the domain of crossfit workouts is less common than the others) and C9 (the most complex test case).

Discussion. The average F1 scores of nodes and arcs are 0.57 and 0.24, respectively, whose average is 0.41. The answer to RQ.3, "How does ChatGPT perform in supply-driven design?", is then **"poor"**. A comparison of the schemata obtained with the corresponding ground truths shows that the most frequent errors made by ChatGPT are:

1. *Missing FDs*. Arcs that are missing because the corresponding FD has not been detected.
2. *Reverse FDs*, i.e., arcs that should be present but in the opposite direction.
3. *Wrong fact*. In this situations, ChatGPT failed to correctly pick the fact as the source table from which most of the sources schema can be covered.
4. *Missing measures*, i.e., one or more of the measures in the ground truth were not classified as such.
5. *YAML syntax*. In this case, ChatGPT either included in the output some additional tag (e.g., "dimensions") or created a wrong indentation (which invalidates the YAML code).

3.5. Answer to RQ.4: Improved supply-driven design

To answer RQ.4 we engineered two different instruction prompts based, respectively, on a declarative and an algorithmic procedure and including three examples of its application. In both cases, we crafted the prompt via an iterative approach in which we progressively added specific sentences to the procedure to try to fix the errors found at the previous iteration.

Fig. 6. A DFM schema created by ChatGPT in supply-driven mode (with no prompt engineering) for C1 (top) and the corresponding ground truth (bottom); dimensions are shown in cyan. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

1. Let S be the relational source schema. The main idea of the declarative procedure is to ask ChatGPT to create a connected graph G whose nodes and arcs correspond, respectively, to the attributes of S and to the FDs expressed by S. Then, it is suggested to find the roots of G and choose as the fact the root of the largest tree. Finally, it is suggested that numerical attributes are labeled as measures.
2. The algorithmic procedure is an attempt to teach ChatGPT the FD-chasing algorithm described in [8], which creates G as follows (an excerpt is reported in Section 3.1.5):

   (i) FIND A FACT, i.e., a relation R that has some temporal or date attributes and includes some foreign keys.
   (ii) INITIALIZE: create the root of G as the primary key of R, $R.K$.
   (iii) EXPAND R: add to G an arc from $R.K$ to each other attribute of R that is not a foreign key. For each foreign key $R.F$ in R that references the primary key $S.K$ of another relation S, add to G an arc from $R.K$ to $S.K$ then EXPAND S. Repeat until no more arcs can be added.
   (iv) REARRANGE G, by adding FDs that can be inferred from the application domain, if any (e.g., region → state).
   (v) RENAME THE FACT, i.e., rename the root of the tree as the name of the relation chosen as a fact, R.
   (vi) CHOOSE MEASURES, i.e., label as measures the numerical attributes.

The results are summarized in Fig. 7. The worst results are those of C7, for two main reasons: (i) the fact is not chosen as the one maximizing the coverage of the source schema, and (ii) the shared hierarchies are not correctly recognized.

Discussion. The average F1 scores of nodes and arcs are 0.90 and 0.72, respectively, for the algorithmic approach; 0.88 and 0.67 for the declarative approach. The metrics returned by the declarative and algorithmic procedures are similar, meaning that the two procedures are equally effective, although the algorithmic one is slightly superior for all test cases. Overall, taking the algorithmic approach as a reference, the average F1 score is 0.81. The answer to RQ.4, "How does ChatGPT perform in supply-driven design when prompt engineering is adopted?", is then "**good**". Considering that in RQ.3 the basic performance of ChatGPT has been rated as poor, we conclude that, based on our tests, a careful prompt engineering can significantly improve the accuracy of supply-driven design. The main residual errors are related to (i) missing arcs, which create some fragmentation in the DFM schema; (ii) non-recognition of shared hierarchies. In most cases the fact is correctly chosen, except for a couple of execution of test case C5 (where another table expressing a many-to-many relationship is picked) and for all executions of C7 (as already stated).

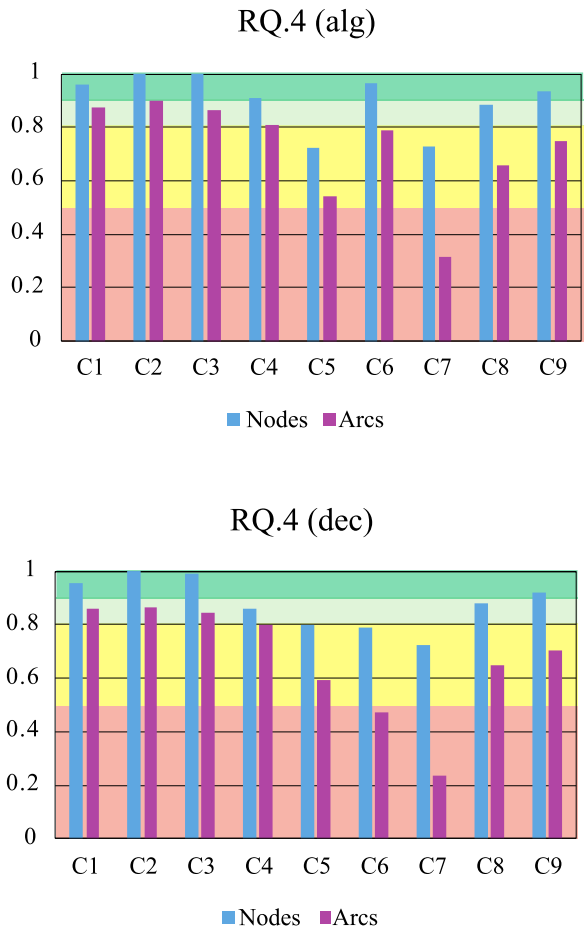

Fig. 7. F1 score for nodes and arcs of the DFM schemata obtained for each test case (improved supply-driven design; top: algorithmic procedure; bottom: declarative procedure).

3.6. Answer to RQ.5: Demand-driven design

To understand what ChatGPT can actually do in demand-driven design, we fed it with all the test cases summarized in Table 1. To this end we use, for each test case, a simple instruction prompt (no specific procedure, no example, just a request to carry out demand-driven design) followed by a case prompt specifying a set of requirements expressed in natural language:

> **Prompt (excerpt)**: Carry out demand-driven conceptual design of a multidimensional cube using the DFM (Dimensional Fact Model), starting from the following requirements.
>
> Decision-makers are interested in analyzing the quantity, revenue, and unit price of purchases of articles made in stores on a day-by-day basis. A store has a name, an address, and is located in a region of a state (each state is part of one country). Articles are described by their names and their types.
>
> The output I expect is a DFM schema in YAML formatted as follows: [...]. Return only the YAML without any further information or explanation.

The results are summarized in Fig. 8, in terms of the accuracy metrics introduced in Section 3.1.7. As an example, Fig. 9 allows to compare a DFM schema created by ChatGPT with the ground truth for test case C7. ChatGPT correctly identifies the fact and the measures but (i) fails to recognize the shared hierarchies in Date and Office; (ii) misses the Risk dimension; (iii) creates an additional Car attribute (should be CarPlate, which in the requirements is stated to be a unique identifier for cars); and (iv) draws some hierarchy arcs in the wrong direction. C8 is the test case that gives the worst results in terms of arcs for different reasons: (i) sometimes the fact is confused with a dimension, (ii) some shared hierarchies are not recognized, and (iii) some missing FDs are not detected.

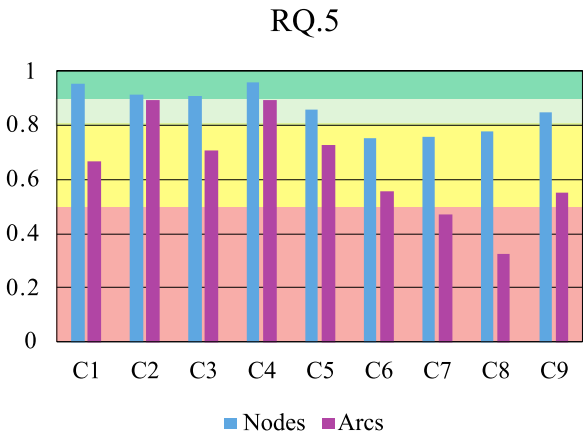

Fig. 8. F1 score for nodes and arcs of the DFM schemata obtained for each test case (demand-driven design).

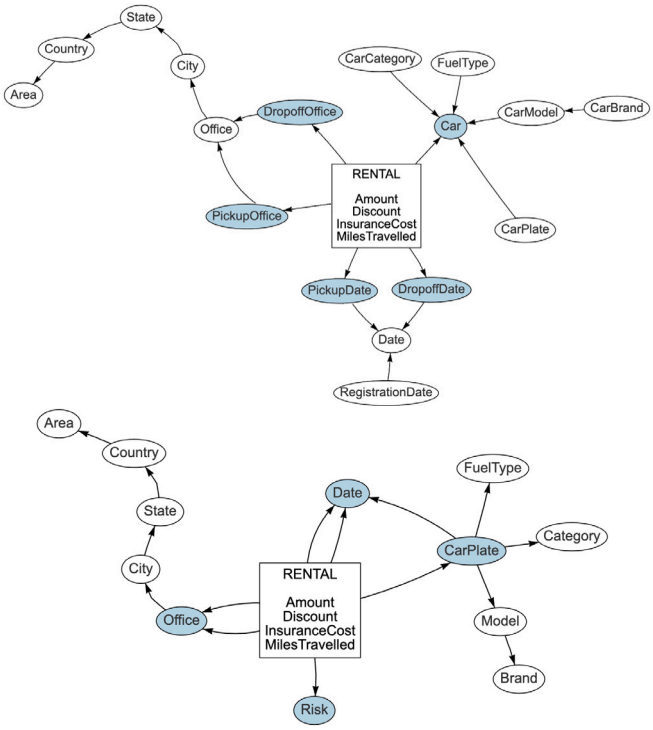

Fig. 9. A DFM schema created by ChatGPT in demand-driven mode (with no prompt engineering) for C7 (top) and the corresponding ground truth (bottom).

Discussion. The average F1 scores of nodes and arcs are 0.86 and 0.64, respectively, whose average is 0.75. The answer to RQ.5, "How does ChatGPT perform in demand-driven design?", is then **"average"**. A comparison of the schemata obtained with the corresponding ground truths shows that the most frequent errors made by ChatGPT in this case are:

1. *Reverse FDs*.
2. *Missing FDs*.
3. *Wrong dimensions*. In this situations, ChatGPT failed to correctly identify one or more concepts expressed in the requirements as dimension.
4. *Shared hierarchies* not recognized, for instance, shipment date and delivery date represented as two distinct attributes.

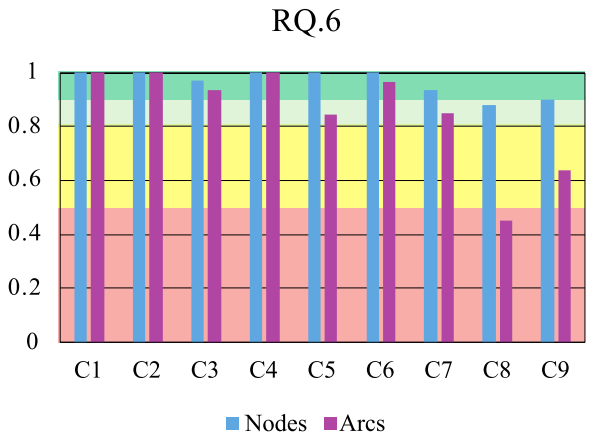

Fig. 10. F1 score for nodes and arcs of the DFM schemata obtained for each test case (improved demand-driven design).

3.7. Answer to RQ.6: Improved demand-driven design

As done for RQ.4, to answer RQ.6 we engineered a prompt aimed at fixing the errors most commonly made by ChatGPT in demand-driven design. More precisely:

> **Prompt (excerpt)**: A DFM is a connected graph where the fact is a node in which no arcs enter. The fact is a business phenomenon the users wish to analyze. The arcs exiting the fact lead either to measures (always numerical) or to dimensions (always categorical). The arcs within a hierarchy are always directed from the finer attribute towards the coarser one (e.g., date → month → year). This means that there is an arc from attribute a to attribute b when one member of a is related to one member of b, while one member of b is related to many members of a. When you have the same concept used twice or more (e.g., "delivery date" and "shipment date"), instead of keeping these attributes separate, you should create two or more arcs reaching a single node ("date") and add roles to the arcs ("delivery" and "shipment"). When the requirements specify that a concept is identified by an attribute, do not create a node for the concept but only for the identifier.

Besides, three examples are provided according to the few-shot learning and chain-of-thought approach.

The results are summarized in Fig. 10, and show a clear improvement over those of RQ.5. The worst test case is still C8, again for a wrong naming of the fact.

Discussion. The average F1 scores of nodes and arcs are 0.96 and 0.85, respectively, whose average is 0.91. The answer to RQ.6, "How does ChatGPT perform in demand-driven design when prompt engineering is adopted", is then **"excellent"**. Considering that in RQ.5 the basic performance of ChatGPT has been rated as average, we conclude that, as for supply-driven design, a careful prompt engineering can significantly improve the accuracy of demand-driven design. In a few cases, ChatGPT even introduces a useful concept not mentioned in the requirements, thus improving over the ground truth we provided (for instance, in C5, it sometimes introduces the concept of session). The residual errors are mainly related to the non-recognition of shared hierarchies and of unique identifiers mentioned in the requirements.

3.8. Answer to RQ.7: Dependence of demand-driven design on requirements formulation

To answer this RQ, we asked three users knowledgeable about the DFM, U1, U2, and U3, to write the requirements for two representative test cases, C3 and C7, starting from the DFM schema used as ground truth. Clearly, they were not shown the original requirements formulation, i.e., the one we had used for the other tests. To get an additional formulation, we also asked ChatGPT to write a natural language requirements specification by reverse engineering, i.e., starting from the DFM schemata corresponding to the ground truth of these two test cases. An example of how these formulations differ from the original one (by U0, used to answer RQ.6) is given in Fig. 11 for test case C3.

Then, we gave ChatGPT the same improved prompt used in RQ.6, coupled with the new formulations. The results (averaged on 10 trials) are shown in Fig. 12 (bars U0 refer to the original formulation used for RQ.6).

Discussion. The first observation is that the requirements formulation created by ChatGPT via reverse engineering, while much more verbose than the others, is not evidently better in term of accuracy of the results — in fact, for C3 it is the worse one. As to the formulations given by the other three users, there seems to be no clear correlation between the "style" adopted to write the requirements and the accuracy (especially for U3, whose formulation is the best one for C3 and the worse one for C7). This is confirmed by executing the ANOVA test on the F1 scores, based on which we cannot reject the null hypothesis (i.e, there is no evidence that the writing style has an impact on the accuracy of ChatGPT). The answer to RQ.7, "Does the performance in demand-driven design depend on how the end-user requirements are expressed?", is then **"no"**.

U0: Decision-makers are interested in analyzing the amount and exchange of purchases made in stores using credit cards on a day-by-day basis. A store has a name, an address, a type, and is located in a country of an area. Credit cards have a type, a holder, an expiration date, and a credit limit. Each card type is described by a circuit, an issuing bank (with its address and country), by a card name, and by a color. A holder has a first name, a last name, an age, a job, and an income range.

U1: As a digital payments company, we want to build a system to extract analytics about our credit cards and specifically the purchases that involve them. Purchases are made by credit card holders in different stores all over the world. For every purchase, we have information about the store in which it is done and for the payment transaction. In this way, we know the date, amount and exchange rate used. For stores, we know the name, type, address and we keep track of the country where it is located along with the geographic area. The holders of a credit card can do various purchases until the credit limit is reached. We are careful to maintain information about holders, in particularly first name, last name, age, job and income. Our cards are of different types related to the circuit, color of the card, name associated and the issuing bank. The banks are international and we have the address and country for each one.

U2: I need to store the amount of purchases done in stores characterized by names, addresses, types, and countries; countries are also grouped in areas of interest. On a given date, purchases are paid through multi-currency credit cards, so I also need to record the exchange rate for each purchase. For a credit card, I am interested in storing the credit limit, type, circuit, color, name, and the issuing bank along with its address and country. A card also has an expiration date and a holder of which I know the holder's name, last name, age, income range, and job.

U3: A company stores are organized in areas which are dislocated across different countries, where each country belongs to a single area. Each store is characterized by a store type, a name, address, and the country it belongs to. Purchases within any store are made via credit cards can be described by the date, amount paid, exchange value and credit card informations. Each credit card has a type associated, which determines the circuit, color of the card, a credit card name and the issuing bank. For each issuing bank its address and country are stored. A credit card also entails a credit limit and is associated to a credit card holder, for whom first name, last name, age, job and income ranges are store.

ChatGPT: Our goal is to understand and analyze purchases made by customers. Every time a purchase happens, we want to capture important details about the transaction, such as where it took place, how it was paid for, and who the customer was. Each purchase includes: The total amount spent; The exchange rate at the time of purchase (if applicable); Where the Purchase Happened. We need to know details about the store where the purchase took place, including: The name of the store; Its address;, The type of store (e.g., supermarket, clothing store, online shop); The country where the store is located; The region or area of the country; How the Purchase Was Paid For. Since purchases are made using credit cards, we need to know: The type of credit card used (e.g., Visa, Mastercard); Who owns the credit card (the cardholder); When the card expires; The card's credit limit (how much the customer can spend on it). We also want to know more about the credit card itself, such as: The bank that issued it; The payment network it belongs to (e.g., Visa, Mastercard); The card's branding details, like its color and product name. The issuing bank should also be identified, including: Its name; Its address; The country where it operates; Who Made the Purchase. To better understand our customers, we want to know: Their first and last name; Their age; Their job; Their income range (general financial category); When the Purchase Happened. Every purchase has a date, which helps us track buying trends over time. We also need to keep track of the expiration date of the credit card. With all this information, we can answer important business questions, such as: Which stores generate the most sales? How do customer demographics (age, income, job) influence spending? Which banks issue the most used credit cards? Are there seasonal trends in purchases?

Fig. 11. Alternative formulations of natural language requirements for C3.

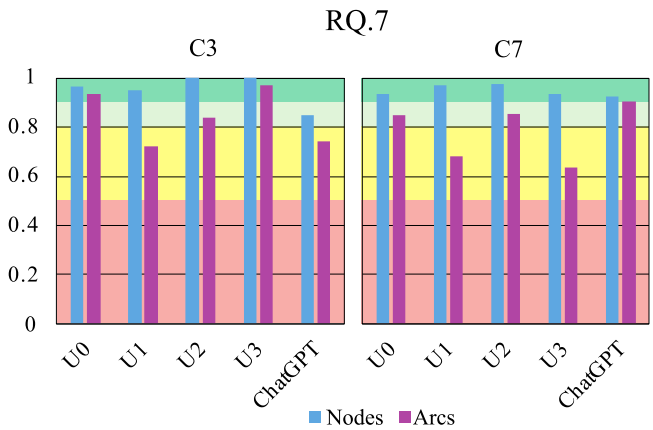

Fig. 12. F1 score for nodes and arcs of the DFM schemata obtained for C3 and C7 with different formulations of requirements (improved demand-driven design).

3.9. Answer to RQ.8: Nodes vs. arcs accuracy

Table 2 shows the average F1 scores of nodes and arcs for the five main RQs.

Discussion. It clearly appears that, in all the tests we made, the F1 score of nodes is significantly higher than the one of arcs. This is not surprising, since exactly capturing all the attribute names present in the source schema or in the natural language requirements is relatively easy, while correctly modeling the FDs entailed by foreign keys or expressed in the text may be hard (even for a designer). The answer to RQ.8, "Does ChatGPT perform better when designing nodes or arcs of the DFM schema?", is then **"nodes"**.

Table 2
Average F1 scores of nodes and arcs for the different RQs.

Approach	RQ	F1 score (nodes)	F1 score (arcs)	F1 score (avg)
Supply-driven	RQ.3	0.57	0.24	0.41
	RQ.4 (alg)	0.90	0.72	0.81
	RQ.4 (dec)	0.88	0.67	0.78
Demand-driven	RQ.5	0.86	0.64	0.75
	RQ.6	0.96	0.85	0.91

3.10. Answer to RQ.9: Supply-driven vs. demand-driven design

Table 2 shows that, both when simple and improved prompts are used, demand-driven design outperforms supply-driven design. The answer to RQ.9, "Does ChatGPT perform better in supply-driven or in demand-driven design?", is then **"demand-driven"**.

3.11. Threats to validity

Validity threats are often grouped into four categories: conclusion, internal, construct, and external validity [41].

- One possible conclusion threat is related to the reliability of measures. Our evaluation uses F1 scores, which in turn are based on precision and recall for node and arc matching. This metric is strictly related to the structure of the graph and to the role of nodes (e.g., a measure would not match with an attribute even if their name is the same); to avoid introducing subjectivity, it does not weigh errors based on their perceived seriousness. For instance, one could argue that completely missing a FD is a more serious error than adding the FD in the wrong direction — but this would be subjective. Another conclusion threat is related to the low statistical power of the test; to cope with this, we defined test cases with increasing sizes and different types of complexity (see Table 1). Besides, for each research case and each test case, the prompts were submitted to ChatGPT 10 times and the results were averaged.
- The main internal threat could be related to history, i.e, different treatments applied to the same object at different times. To cope with this, as stated in Section 3.1.2, we started a new chat for each repetition of each case.
- The main construct threat we suffer from is a design threat related to demand-driven design, specifically, to the expression of the requirements in natural language and to the creation of the corresponding ground truth. Indeed, both tasks were executed by expert data warehouse designers, who expressed the requirements in a clear and non-ambiguous way (presumably, more precisely than how a business user would do). The experiments with supply-driven design do not suffer from this threat since the ground truth could be derived algorithmically.
- Finally, many threats to external validity are caused by an artificial setting of the experiment [42], specifically, by unrealistic tasks. To cope with this, we paid a lot of attention to the preparation of the test cases for the experiment, choosing realistic domains with realistic descriptions.

4. Conclusion

In this paper we have investigated the capabilities of ChatGPT to cope with supply-driven and demand-driven conceptual design of multidimensional data cubes, using the DFM as a reference model. It turned out that, although ChatGPT apparently tends to mix the conceptual level (DFM) with the logical level (star/snowflake schemata), once it has been instructed on how to represent a DFM schema using YAML, it can provide some results at different degrees of accuracy using simple prompts. However, when prompts are enhanced with detailed procedural instructions and examples, the produced results significantly improve in all cases. Overall, ChatGPT is better at demand-driven design than at supply-driven design; this is not surprising, since demand-driven design is more focused on an interpretation of natural language than on the execution of an algorithm. It also appears that ChatGPT tends to cope better with nodes (attributes and measures) than with arcs (FDs), especially in supply-driven design. Finally, an algorithmic procedure works slightly better than a declarative one in instructing ChatGPT on supply-driven design.

Despite the encouraging results obtained, some limitations of using an LLM for multidimensional conceptual design have clearly emerged during research. We list them in the following, together with some possible mitigations:

1. *Processable size*. Due to the limitations in the number of input tokens, it is currently not possible to handle very large source schemata (such as those commonly found in real projects) and to give ChatGPT comprehensive and detailed instructions on how to deal with all possible design situations. A mitigation could be to specifically train an LLM on multidimensional design, so that it already incorporates these concepts. Another possibility would be to check if the recently proposed open-source solutions can compete with ChatGPT [6,12,16,43,44].
2. *Task complexity*. LLMs are recognized to struggle with complex modeling that requires specific modeling knowledge besides the domain description. This is confirmed by our tests which show that, although precise instructions are given to ChatGPT about how to carry out supply-driven design, its performance is worse than the one for demand-driven design. In this work we have adopted the chain-of-thought principle to try to mitigate this issue [44]; moving to the *tree-of-thought* principle could be a further mitigation [28]. Directly injecting the knowledge about multidimensional modeling into ChatGPT is another promising direction [25].

3. *Domain knowledge*. The more ChatGPT knows about the domain in demand-driven design, the more accurate the DFM schema it can generate, even if the requirements are blurry and incomplete. Indeed, ChatGPT appeared to lack domain-specific knowledge for some test cases. Possible mitigations of this issue could be (i) making a catalogue of common multidimensional modeling solutions from different domains publicly available, (ii) devising incremental prompting strategies that involve the end-user to verify and refine the requirements, and (iii) fine-tuning the model on domain-specific literature [12,13,45].
4. *User interaction*. As shown in Fig. 4, ChatGPT cannot directly generate reliable graphical representations of a DFM schema. Thus, additional platforms must be used to generate graphical outputs as the final deliverables. Together with issues such as the lack of semantic understanding and the variability of results due to non-determinism, this makes the user interaction with ChatGPT unnatural and may undermine the effectiveness of conceptual design. Possible mitigations are (i) the inclusion of assistive features such as automatic graphical formatting for schemata, multi-user collaboration capabilities, and voice-mediated interaction, (ii) continuous follow-up questioning to keep conceptual schemata aligned with evolving end-user needs, and (iii) introducing design-specific templates and guidelines [10–12,16,43].
5. *Correctness of resulting schemata*. Despite the adoption of improved reasoning strategies such as chain-of-thought, hallucinations may still be generated in the DFM schemata due to inconsistencies in the decomposed reasoning process. Two possible solutions to alleviate this problem are (i) to include scripts that code some algorithmic parts of the task into the design process and (ii) to integrate process-based supervision during training [43,44].

Besides these limitations, our future work will also address the following specific issues:

- Evaluate the improvements obtained when an iterative approach to prompting is adopted to repair the errors made by ChatGPT (especially when the schema delivered in output is non-connected).
- Check if the *Requirements Simulator Pattern* defined in [3] can be applied to let end-users verify that a DFM schema (especially if obtained via demand-driven design) actually meets their requirements.
- Extend the experiments to cover the advanced constructs of the DFM (e.g., descriptive attributes, additivity, irregular hierarchies), also introducing test cases relying on geospatial data — which usually pose additional challenges to LLMs.
- Evaluate how ChatGPT can support end-users who refine the draft schemata obtained via supply-driven engineering to have them meet their analysis requirements (e.g., by removing non-interesting attributes or adding FDs). A preliminary step in this direction has been done in [46].

CRediT authorship contribution statement

Stefano Rizzi: Writing – review & editing, Writing – original draft, Methodology, Investigation, Conceptualization. **Matteo Francia:** Writing – review & editing. **Enrico Gallinucci:** Writing – original draft, Validation, Software. **Matteo Golfarelli:** Writing – review & editing, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

[1] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Trans. Softw. Eng. Methodol. 33 (8) (2024) 220:1–220:79.
[2] W. Ma, S. Liu, W. Wang, Q. Hu, Y. Liu, C. Zhang, L. Nie, Y. Liu, LLMs: Understanding code syntax and semantics for code analysis, 2023, CoRR abs/2305.12138.
[3] J. White, S. Hays, Q. Fu, J. Spencer-Smith, D.C. Schmidt, ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design, 2023, CoRR abs/2303.07839.
[4] H. Fill, J. Cabot, W. Maass, M. van Sinderen, AI-driven software engineering - the role of conceptual modeling, Enterp. Model. Inf. Syst. Arch. Int. J. Concept. Model. 19 (2024).
[5] L. Wang, W. Shen, H. Xie, J. Neelamkavil, A. Pardasani, Collaborative conceptual design - state of the art and future trends, Comput. Aided Des. 34 (13) (2002) 981–996.
[6] H. Fill, P. Fettke, J. Köpke, Conceptual modeling and large language models: Impressions from first experiments with ChatGPT, Enterp. Model. Inf. Syst. Arch. Int. J. Concept. Model. 18 (2023) 3.
[7] R. Lutze, K. Waldhör, Generating specifications from requirements documents for smart devices using large language models (LLMs), in: Proc. HCI, Washington, DC, USA, 2024, pp. 94–108.
[8] M. Golfarelli, S. Rizzi, Data Warehouse Design: Modern Principles and Methodologies, McGraw-Hill, 2009.
[9] W. Hariri, Unlocking the potential of ChatGPT: A comprehensive exploration of its applications, advantages, limitations, and future directions in natural language processing, 2023, CoRR abs/2304.02017.

[10] Z. Zhou, J. Li, Z. Zhang, J. Yu, H. Duh, Examining how the large language models impact the conceptual design with human designers: A comparative case study, Int. J. Hum. Comput. Interact. (2024) 1–17.
[11] B. Wang, C. Wang, P. Liang, B. Li, C. Zeng, How LLMs aid in UML modeling: An exploratory study with novice analysts, in: Proc. SSE, Shenzhen, China, 2024, pp. 249–257.
[12] J. Cámara, J. Troya, L. Burgueño, A. Vallecillo, On the assessment of generative AI in modeling tasks: an experience report with ChatGPT and UML, Softw. Syst. Model. 22 (3) (2023) 781–793.
[13] K. Ma, D. Grandi, C. McComb, K. Goucher-Lambert, Conceptual design generation using large language models, in: Proc. IDETC-CIE, Boston, MA, US, 2023, pp. 1–21.
[14] L. Chen, Y. Tsang, Q. Jing, L. Sun, A LLM-augmented morphological analysis approach for conceptual design, in: Proc. DRS, Boston, USA, 2024, pp. 1–19.
[15] S.J. Ali, I. Reinhartz-Berger, D. Bork, How are LLMs used for conceptual modeling? an exploratory study on interaction behavior and user perception, in: Proc. ER, Pittsburgh, USA, 2024, pp. 257–275.
[16] F. Muff, H. Fill, Limitations of ChatGPT in conceptual modeling: Insights from experiments in metamodeling, in: Proc. Modellierung, Potsdam, Germany, 2024, p. 8.
[17] A.D. Conrardy, J. Cabot, From image to UML: First results of image-based UML diagram generation using LLMs, in: Proc. LLM4MDE, Enschede, The Netherlands, 2024, pp. 55–65.
[18] O. Romero, A. Abelló, Data-driven multidimensional design for OLAP, in: Proc. SSDBM, Portland, OR, USA, 2011, pp. 594–595.
[19] P. Jovanovic, O. Romero, A. Simitsis, A. Abelló, D. Mayorova, A requirement-driven approach to the design and evolution of data warehouses, Inf. Syst. 44 (2014) 94–119.
[20] F.D. Tria, E. Lefons, F. Tangorra, Hybrid methodology for data warehouse conceptual design by UML schemas, Inf. Softw. Technol. 54 (4) (2012) 360–379.
[21] O. Romero, A. Abelló, Automatic validation of requirements to support multidimensional design, Data Knowl. Eng. 69 (9) (2010) 917–942.
[22] T. Niemi, J. Nummenmaa, P. Thanisch, Constructing OLAP cubes based on queries, in: Proc. DOLAP, Atlanta, Georgia, USA, 2001, pp. 9–15.
[23] S. Bimonte, L. Antonelli, S. Rizzi, Requirements-driven data warehouse design based on enhanced pivot tables, Req. Eng. 26 (1) (2021) 43–65.
[24] R. Nair, C. Wilson, B. Srinivasan, A conceptual query-driven design framework for data warehouse, Int. J. Comput. Inf. Eng. 1 (1) (2007) 62–67.
[25] K. Chen, Y. Yang, B. Chen, J.A.H. López, G. Mussbacher, D. Varró, Automated domain modeling with large language models: A comparative study, in: Proc. MODELS, Västerås, Sweden, 2023, pp. 162–172.
[26] T.B. Brown, et al., Language models are few-shot learners, in: Proc. NeurIPS, 2020, pp. 1877–1901.
[27] J. Wei, et al., Chain-of-thought prompting elicits reasoning in large language models, in: S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, A. Oh (Eds.), Proc. NeurIPS, New Orleans, LA, USA, 2022, pp. 24824–24837.
[28] J. Silva, Q. Ma, J. Cabot, P. Kelsen, H. Proper, Application of the tree-of-thoughts framework to LLM-enabled domain modeling, in: Proc. ER, Pittsburgh, USA, 2024, pp. 94–111.
[29] D. Prokop, Š. Stenchlák, P. Škoda, J. Klímek, M. Nečaský, Enhancing domain modeling with pre-trained large language models: An automated assistant for domain modelers, in: Proc. ER, Pittsburgh, USA, 2024, pp. 235–253.
[30] R. Kimball, The Data Warehouse Toolkit, John Wiley & Sons, 1996.
[31] V. Jovanovic, I. Bojicic, Conceptual data vault model, in: Proc. SAIS, Vol. 23, Atlanta, Georgia, 2012, pp. 1–6.
[32] H. Hultgren, Data vault modeling guide, 2012, hanshultgren.files.wordpress.com.
[33] D. Krneta, V. Jovanovic, Z. Marjanovic, A direct approach to physical data vault design, Comput. Sci. Inf. Syst. 11 (2) (2014) 569–599.
[34] L. Rönnbäck, O. Regardt, M. Bergholtz, P. Johannesson, P. Wohed, Anchor modeling - agile information modeling in evolving data environments, Data Knowl. Eng. 69 (12) (2010) 1229–1253.
[35] W. Inmon, Corporate Information Factory 2E, John Wiley & Sons, 2000.
[36] C. Sapia, M. Blaschka, G. Höfling, B. Dinter, Extending the E/R model for the multidimensional paradigm, in: Proc. DWDM, 1998, pp. 105–116.
[37] J. Trujillo, M. Palomar, An object-oriented approach to multidimensional database conceptual modeling, in: Proc. DOLAP, Bethesda, Maryland, USA, 1998, pp. 16–21.
[38] A. Abelló, J. Samos, F. Saltor, Benefits of an object-oriented multidimensional data model, in: Proc. Int. Symp. on Objects and Databases, Sophia Antipolis, France, 2000, pp. 141–152.
[39] B.T. Nguyen, A.M. Tjoa, R.R. Wagner, An object oriented multidimensional data model for OLAP, in: Proc. WAIM, Shanghai, China, 2000, pp. 69–82.
[40] A. Gosain, S. Mann, Empirical validation of metrics for object oriented multidimensional model for data warehouse, Int. J. Syst. Assur. Eng. Manag. 5 (3) (2014) 262–275.
[41] C. Wohlin, P. Runeson, M. Höst, M.C. Ohlsson, B. Regnell, A. Wesslén, Experimentation in Software Engineering, Springer Publishing Company, 2012, Incorporated.
[42] D. Sjøberg, B. Anda, E. Arisholm, T. Dybå, M. Jørgensen, A. Karahasanovic, E. Koren, M. Vokác, Conducting realistic experiments in software engineering, in: Proc. ISESE, Nara, Japan, 2002, pp. 17–26.
[43] L. Netz, J. Michael, B. Rumpe, From natural language to web applications: Using large language models for model-driven software engineering, in: Proc. Modellierung, Potsdam, Germany, 2024, pp. 179–195.
[44] W.X. Zhao, et al., A survey of large language models, 2023, CoRR abs/2303.18223.
[45] A. Ferrari, S. Abualhaija, C. Arora, Model generation with LLMs: From requirements to UML sequence diagrams, in: Proc. RE, Reykjavik, Iceland, 2024, pp. 291–300.
[46] S. Rizzi, Using ChatGPT to refine draft conceptual schemata in supply-driven design of multidimensional cubes, in: Proc. DOLAP, Barcelona, Spain, 2025, pp. 64–68.

Stefano Rizzi is Full Professor at the University of Bologna, Italy. He has authored more than 150 papers in international journals and conferences mainly in the fields of data warehousing and business intelligence. He is member of the steering committee of DOLAP and has been a member of the editorial board of DKE and of the steering committee of the ER Conference. His research interests include data warehouse design and business intelligence, in particular OLAP on NoSQL data and analysis services for big data.

Matteo Francia is Assistant Professor at the University of Bologna, Italy. His research focuses on machine learning and advanced big data analytics, with particular reference to mobility data and precision agriculture. He served as associate editor for the DKE Journal, as guest editor for several international journals, and as program co-chair at the DATAPLAT Workshop. At EDBT 2021 he received the best demo award.

Enrico Gallinucci is Assistant Professor at the University of Bologna, where he received his Ph.D. in Computer Science and Engineering and teaches Business Intelligence and Big Data. His research interests currently focus on big data analytics, NoSQL and multimodel database systems, data democratization, and precision agriculture. He is associate editor for the DKE journal.

Matteo Golfarelli is Full Professor at the University of Bologna teaching Information Systems and Data Mining. His researches in the Business Intelligence area covered most of the design issues related to Data Warehouse systems. He is co-author of the book Data Warehouse Design: Modern Principles and Methodologies. His current research interests include Business Intelligence on non-conventional data and Big Data Analytics.